

- ✓ Real-World
- ✓ Production Focused
- ✓ Engineer Approved
- ✓ Interview Ready

VERIQTA

Learn. Build.
Deploy. Repeat.

KUBERNETES

ADVANCED ARCHITECTURE

VOLUME

3

From
Production
Incidents to
Platform
Excellence

A Practical Guide to

Designing, Securing and Operating
Enterprise Grade Kubernetes Platforms



Inside This Volume

- ✓ Advanced Networking
- ✓ Security & Governance
- ✓ Operators & CRDs
- ✓ Service Mesh
- ✓ Multi-Cluster
- ✓ GitOps & Platform
- ✓ Architecture Patterns
- ✓ Real-World Examples
- ✓ Cheat Sheets & Checklists



Advanced
Concepts

Deep dive into
enterprise patterns



Secure
by Design

Build resilient &
secure platforms



Platform
Ready

Operate at scale
with confidence



Practical
Examples

Real YAML, diagrams
& best practices



Checklists
& Cheats

Quick references for
daily operations

“ Architecture is about the important stuff.
Everything else is detail.”

1. INGRESS

"The smart gatekeeper for your Kubernetes services."

1 WHAT IS INGRESS?

- Ingress exposes HTTP and HTTPS routes from outside the cluster to services inside the cluster.
- It provides a single entry point for multiple services.
- Ingress does NOT expose ports or load balance by itself.
- Requires an Ingress Controller to work.

KEY BENEFITS

- ✓ Single Entry Point
- ✓ Host & Path Routing
- ✓ TLS/SSL Termination
- ✓ Traffic Management
- ✓ Security & Control
- ✓ Cost Efficient

2 INGRESS CONTROLLER

- An Ingress Controller is a specialized proxy/load balancer that watches Ingress resources and implements the routing rules.
- Examples: NGINX Ingress, Traefik, HAProxy, Envoy, etc.

POPULAR CONTROLLERS

- NGINX Ingress
- Traefik
- HAProxy Ingress
- AWS ALB Ingress
- GCP HTTP(S) LB
- Istio Gateway (Mesh)

3 HTTP ROUTING

- Ingress routes HTTP/HTTPS traffic based on rules defined in Ingress resources.
- Supports host-based and path-based routing.

PROTOCOLS SUPPORTED

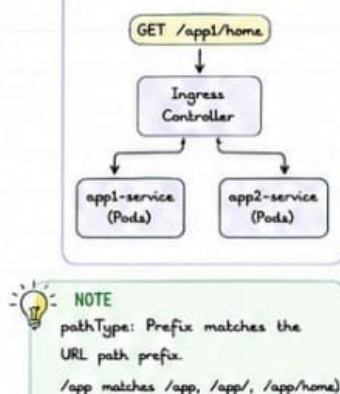
HTTP
HTTPS
gRPC (via annotations)
WebSocket

4 PATH-BASED ROUTING

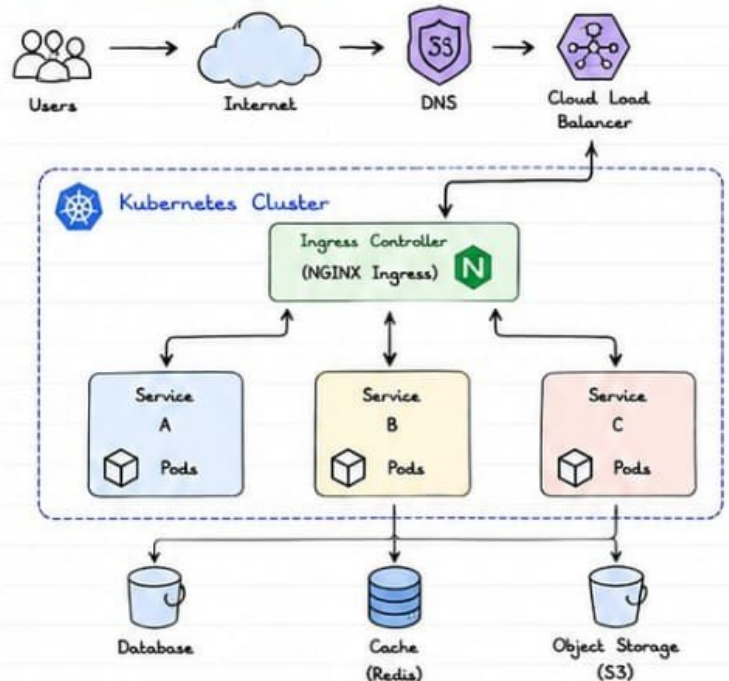
- Routes traffic to different services based on the URL path.

```
rules:
- http:
  paths:
  - path: /app1
    pathType: Prefix
    backend:
      service:
        name: app1-service
        port:
          number: 80
  - path: /app2
    pathType: Prefix
    backend:
      service:
        name: app2-service
        port:
          number: 80
```

EXAMPLE RUNTIME FLOW



6 PRODUCTION ARCHITECTURE



5 HOST-BASED ROUTING

- Routes traffic to services based on the request host (domain).

```
rules:
- host: app1.veriqta.com
  http:
    paths:
    - path: /
      pathType: Prefix
      backend:
        service:
          name: app1-service
          port:
            number: 80
- host: app2.veriqta.com
  http:
    paths:
    - path: /
      pathType: Prefix
      backend:
        service:
          name: app2-service
          port:
            number: 80
```

EXAMPLE

Incoming Requests

app1.veriqta.com /anything

app2.veriqta.com /anything

Routed To

app1-service (Pods)

app2-service (Pods)

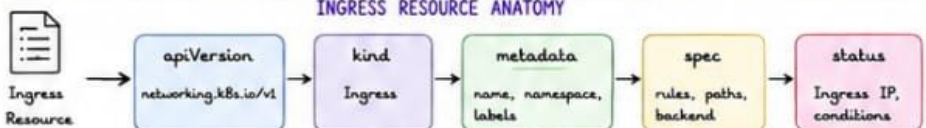


NOTE
pathType: Prefix matches the URL path prefix.
/app matches /app, /app/, /app/home

BEST PRACTICES

- ✓ Use Ingress Controller with HA setup.
- ✓ Enable TLS for all domains.
- ✓ Use host-based routing in production.
- ✓ Monitor Ingress Controller metrics.
- ✓ Keep Ingress rules simple & clear.

INGRESS RESOURCE ANATOMY



"Ingress is the front door. The Ingress Controller is the bouncer."

2. INGRESS CONTROLLERS

"Different controllers. Same goal: Smart traffic into your cluster."

1 WHAT IS AN INGRESS CONTROLLER?

- An Ingress Controller is a specialized reverse proxy /load balancer.
- It watches Ingress resources and configures routing rules.
- It handles Layer 7 (HTTP/HTTPS) traffic and advanced routing.
- Runs inside the cluster or outside (cloud provider).

KEY RESPONSIBILITIES

- ✓ Watch Ingress resources
- ✓ Route external traffic
- ✓ Terminate TLS/SSL
- ✓ Load balance requests
- ✓ Apply routing rules
- ✓ Provide observability

2 NGINX INGRESS



- Most popular open-source Ingress Controller.
- High performance and feature rich.
- Supports annotations for advanced configs.
- Actively maintained by Kubernetes community.

BEST FOR

- ✓ General purpose
- ✓ High performance
- ✓ Large community support

POPULAR FEATURES

- Regex path routing
- Rate limiting
- SSL termination
- Rewrite, Redirect
- Canary deployments

3 TRAEFIK



- Modern, cloud-native Ingress Controller.
- Built-in service discovery for many platforms.
- Automatic HTTPS (Let's Encrypt).
- Simple, dynamic configuration.

BEST FOR

- ✓ Microservices
- ✓ Docker/K8s native
- ✓ Automatic TLS

POPULAR FEATURES

- Automatic TLS
- Dashboard (web UI)
- Dynamic config (no reload)
- Middleware (auth, rate limit)

4 HAProxy INGRESS



- Based on HAProxy (enterprise grade).
- Extremely fast and reliable.
- Supports TCP, HTTP, HTTPS, gRPC.
- Great for high performance environments.

BEST FOR

- ✓ High performance
- ✓ TCP/Layer 4 + Layer 7
- ✓ Enterprise workloads

POPULAR FEATURES

- Advanced load balancing
- ACLs and routing rules
- Stickiness / sessions
- Fine-grained control



PRO TIPS

- ✓ Use Ingress Controller with HA replicas.
- ✓ Enable TLS everywhere.
- ✓ Monitor controller metrics.
- ✓ Keep controller updated.
- ✓ Use annotations carefully.

COMMON ANNOTATIONS (NGINX)

nginx.ingress.kubernetes.io/rewrite-target
nginx.ingress.kubernetes.io/ssl-redirect
nginx.ingress.kubernetes.io/rate-limit
nginx.ingress.kubernetes.io/limit-rps

REMEMBER

Ingress Controller is the traffic cop at the entrance of your cluster. Choose the right one for your workload and team.

5 CLOUD LOAD BALANCERS (MANAGED)



AWS ALB/
NLB



GCP
HTTP(S) LB



Azure
Application
Gateway

- Managed by cloud providers.
- Integrated with Kubernetes (Service + Ingress).
- Highly available, auto-scaling, global reach.

BEST FOR

- ✓ Production at scale
- ✓ Global traffic
- ✓ Minimal operational overhead



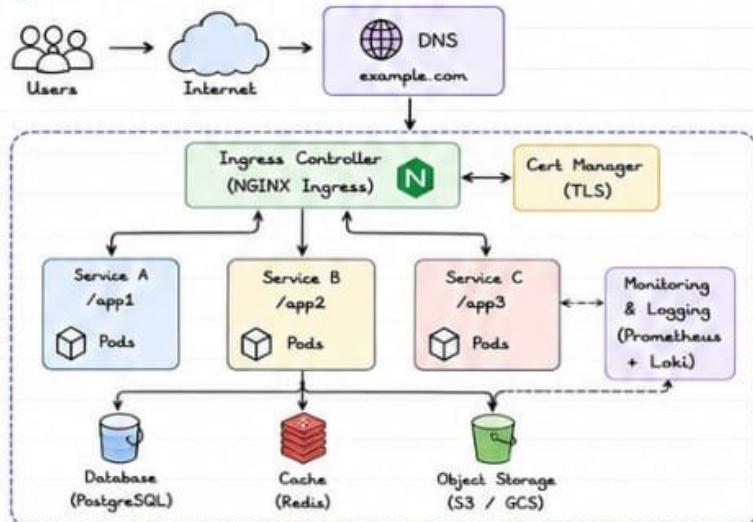
6 CONTROLLER COMPARISON

FEATURE	NGINX	TRAEFIK	HAPROXY	CLOUD LB
Layer	7 (HTTP/HTTPS)	7 (HTTP/HTTPS)	4 & 7	4 & 7
Performance	High	High	Very High	Very High
Ease of Use	Medium	Easy	Medium	Easy
Auto TLS	Via cert-manager	Built-in (Let's Encrypt)	Via cert-manager	Managed
Advanced Routing	Yes	Yes	Yes	Yes
Rate Limiting	Yes	Yes (middleware)	Yes (ACL)	Limited*
Observability	Good	Good	Good	Good
Best For	General Purpose	Modern Microservices	High Perf Enterprise	Managed & Scale



* Cloud LB has limited routing logic compared to Ingress Controllers.

7 PRODUCTION EXAMPLE ARCHITECTURE



"Right controller. Right routing. Right results."

3. GATEWAY API

"The next generation of Kubernetes networking."

1 GATEWAY API OVERVIEW

- Gateway API is the evolution of **Ingress**.
- Standardized by SIG-NETWORK.
- Flexible, extensible and role-oriented.
- Works with multiple implementations (controllers).
- Designed for modern cloud native applications.

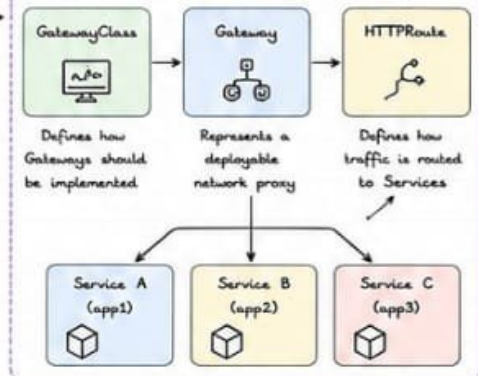
KEY BENEFITS

- ✓ Standardized APIs
- ✓ Better extensibility
- ✓ Advanced traffic mgmt
- ✓ Multi-tenancy ready
- ✓ Cloud & on-prem agnostic
- ✓ Designed for the future

2 GATEWAY API OBJECT MODEL



GATEWAY API RESOURCES



3 GATEWAYCLASS

- Defines a class of gateways.
- Cluster scoped resource.
- Selected by Gateways to determine which controller should implement it.
- Example: nginx, gke-l7-global, aws-lb controller, contours.

EXAMPLE

```

apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: nginx
spec:
  controllerName: k8s.io/ingress-nginx
  parametersRef:
    name: nginx-gateway-params
    group: gateway.nginx.org
    kind: NginxParameters
  
```

4 GATEWAY

- Configures an instance of traffic proxy.
- Listens on addresses and ports.
- Uses a **GatewayClass** for implementation.
- Can expose multiple listeners (HTTP, HTTPS, TCP).
- Can be internal or external.

LISTENER TYPES

- HTTP (port 80)
- HTTPS (port 443)
- TCP
- UDP
- ...

EXAMPLE

```

apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: prod-gateway
spec:
  gatewayClassName: nginx
  listeners:
    - name: web
      protocol: HTTP
      port: 80
    - name: websecure
      protocol: HTTPS
      port: 443
      tls:
        mode: Terminate
        certificateRefs:
          - name: tls-cert
            kind: Secret
  
```

5 HTTPROUTE

- Defines how HTTP traffic is routed.
- Matches based on hostnames, paths, headers, methods.
- Supports advanced rules: filters, rewrites, redirects, timeouts.
- Attaches to a Gateway.

MATCH SUPPORT

- Hostnames
- Paths (Prefix / Exact)
- Headers
- HTTP Methods
- Query Params

💡 Think: Intelligent traffic rules!

EXAMPLE

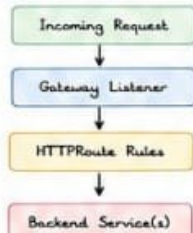
```

apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: app-route
spec:
  parentRefs:
    - name: prod-gateway
      sectionName: web
  hostnames: ["app.veriqta.com"]
  rules:
    - matches:
      - path:
          type: PathPrefix
          value: /app
        backendRefs:
          - name: app-service
            port: 80
    - matches:
      - path:
          type: PathPrefix
          value: /
        backendRefs:
          - name: web-service
            port: 80
  
```

6 TRAFFIC ROUTING CAPABILITIES

CAPABILITY	DESCRIPTION
Path-based routing	Route based on URL path (e.g. /app, /api)
Host-based routing	Route based on domain or subdomain
Header-based routing	Route based on headers (e.g. X-Env)
Traffic splitting	Split traffic across multiple backends
Canary deployments	Send % of traffic to new version
Redirects	HTTP to HTTPS, path rewrites, etc.
Timeouts & retries	Configure timeouts, retries, circuit breaking
Request mirroring	Mirror traffic for testing/observability

RULE FLOW



7 FUTURE OF KUBERNETES NETWORKING

- ✓ Gateway API is becoming the standard for Kubernetes networking.
- ✓ Multi-vendor support (NGINX, Istio, Contour, Traefik, Envoy Gateway).
- ✓ Designed for service mesh integration.
- ✓ Improved extensibility with custom policies & filters.
- ✓ Better support for multi-cluster, edge and enterprise use cases.
- ✓ Will gradually replace Ingress as the primary API.

THE VISION



One API.
Multiple implementations.
Infinite possibilities. ★

GATEWAY API vs INGRESS (QUICK COMPARISON)

INGRESS	ASPECT	GATEWAY API
Kubernetes built-in (beta)	Origin	SIG-NETWORK standard
Single object (Ingress)	API Model	Multiple resources (Class, Gateway, Route)
Limited routing features	Capabilities	Advanced traffic management
Annotations based	Extensibility	Extensible by design
Single controller assumption	Ecosystem	Multi-implementation & vendor neutral
Basic TLS & routing	Enterprise Readiness	Multi-tenancy, security, policies, future-ready

REMEMBER

- ★ GatewayClass → How
- ★ Gateway → Where
- ★ HTTPRoute → What
- ★ Controllers → Who implements



"Gateway API brings **order** to the **chaos** of traffic."

4. NETWORK POLICIES

"Control the traffic in and out of your pods. Default is open. Don't trust. Verify."

1 WHAT IS A NETWORK POLICY?

- Network Policies control traffic flow between pods.
- They are namespace scoped.
- They act like a firewall for pods.
- If no policy applies, traffic is **ALLOWED** by default.
- Once any policy exists, traffic is **DENIED** by default unless allowed.

KEY BENEFITS

- ✓ Zero Trust by default
- ✓ Micro-segmentation
- ✓ Restrict lateral movement
- ✓ Reduce blast radius
- ✓ Compliance ready
- ✓ Better security posture

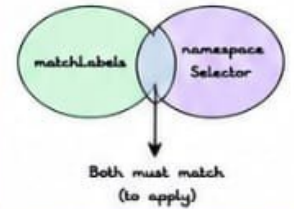
5 LABEL SELECTORS

- Policies use labels to target pods.
- Match pods by key/value pairs.
- Selectors can be combined.
- Use labels consistently.

EXAMPLE SELECTOR

matchLabels:
app: payment
tier: backend

MATCH LOGIC



2 DEFAULT BEHAVIOR

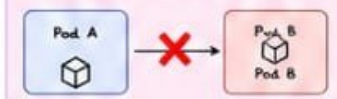
NO NETWORK POLICY

Traffic is open (ALLOW)



AT LEAST ONE POLICY EXISTS

Traffic is blocked (DENY)

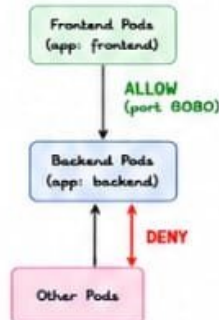


⚠ Important: Kubernetes does not provide network policy engine by default. You need a CNI plugin that supports network policies (Calico, Cilium, Weave, etc).

3 INGRESS RULES

Allow incoming traffic to pods.

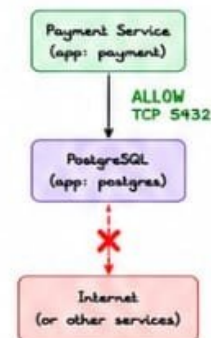
```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-from-frontend
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
      ports:
        - protocol: TCP
          port: 8080
```



4 EGRESS RULES

Control outgoing traffic from pods.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-db-egress
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: payment
  policyTypes:
    - Egress
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              name: db
          podSelector:
            matchLabels:
              app: postgres
      ports:
        - protocol: TCP
          port: 5432
```



6 ZERO TRUST NETWORKING

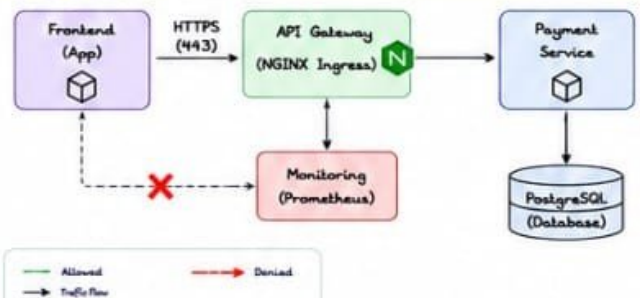
- ✓ Deny all by default.
- ✓ Allow only what is necessary.
- ✓ Segment by namespace, application, environment.
- ✓ Monitor and audit traffic.
- ✓ Assume breach - verify every request.

ZERO TRUST FLOW



7 PRODUCTION EXAMPLE

Scenario: Payment Service



NETWORK POLICY SUPPORT (CNI PLUGINS)

CNI PLUGIN	POLICY SUPPORT	PERFORMANCE	BEST FOR
 Calico	✓ Full (L3/L7)	● High	Production, Enterprise
 Cilium	✓ Full (L3/L7, eBPF)	● Very High	High-scale, eBPF
 Weave Net	○ Basic (L3)	○ Medium	Small Clusters
 AWS VPC CNI	○ Basic (Security Groups)	● High	AWS Native

BEST PRACTICES

- ✓ Always start with deny-all policy.
- ✓ Allow only required traffic.
- ✓ Use namespaces for isolation.
- ✓ Keep rules simple and readable.
- ✓ Monitor with tools (Calico, Cilium Hubble).
- ✓ Review policies regularly.

COMMON MISTAKES

- ✗ Not applying policyTypes (Ingress, Egress).
- ✗ Using wrong labels.
- ✗ Forgetting namespace scoping.
- ✗ Allowing 0.0.0.0/0 everywhere.
- ✗ Ignoring egress (data exfiltration risk).

QUICK COMMANDS

- ❏ `kubectl get networkpolicies -A`
- ❏ `kubectl describe netpol <name> -n <ns>`
- ❏ `kubectl get pods -l app=payment -o wide`
- ❏ `kubectl exec -it <pod> -- curl <service>:<port>`

"Default is open. Your policy is your perimeter."

5. RBAC (ROLE BASED ACCESS CONTROL)

"Right user. Right access. Right resource. Nothing more."

1 USERS

- A user is an identity that can make requests to the Kubernetes API.
- Can be a human or a system.
- Managed outside Kubernetes (not stored in the cluster).
- Authenticated via certs, tokens, OIDC, LDAP, etc.

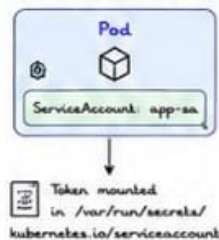
USER EXAMPLES

-  Human (DevOps Engineer)
-  Admin
-  CI/CD System
-  External Application

2 SERVICE ACCOUNTS

- A Service Account is an identity for processes running in pods.
- Automatically mounted in pods.
- Used by applications to access the Kubernetes API.
- Should have minimal permissions.

POD USING SERVICE ACCOUNT



3 ROLES

- Define permissions within a namespace.
- Contain rules that allow or deny actions on resources.
- Use for namespace scoped access.

EXAMPLE: Role (read-pods)

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: read-pods
  namespace: dev
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

CAN DO

- ✓ get pods
- ✓ list pods
- ✓ watch pods

CANNOT DO

- ✗ create pods
- ✗ delete pods
- ✗ access other namespaces

4 CLUSTERROLES

- Define permissions at the cluster level.
- Not limited to a namespace.
- Use for cluster-wide resources or multiple namespaces.

EXAMPLE: ClusterRole (cluster-admin-read)

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cluster-admin-read
rules:
- apiGroups: ["*"]
  resources: ["*"]
  verbs: ["get", "list", "watch"]
```

CAN DO

- Read everything in the cluster
- Across all namespaces

CANNOT DO

- Cannot modify (resources are read-only)

BEST PRACTICES

- ✓ Don't use cluster-admin for daily tasks.
- ✓ Prefer Roles over ClusterRoles.
- ✓ Use ServiceAccounts for workloads.
- ✓ Rotate & review credentials.
- ✓ Audit RBAC regularly.

COMMON MISTAKES

- ✗ Giving cluster-admin to ServiceAccounts.
- ✗ Using default ServiceAccount.
- ✗ Forgetting namespace in RoleBindings.
- ✗ Over-permissioning (more than needed).
- ✗ Not reviewing old bindings.

USEFUL COMMANDS

```
$ kubectl get roles -n <ns>
$ kubectl get rolebindings -n <ns>
$ kubectl get clusterroles
$ kubectl get clusterrolebindings
$ kubectl auth can-i get pods -n dev
--as=system:serviceaccount:dev:app-sa
```

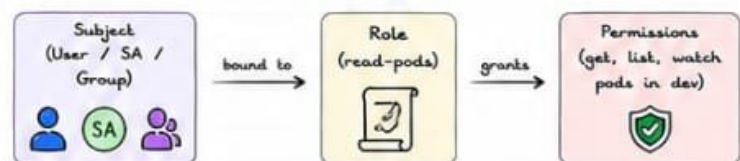
5 ROLEBINDINGS

- Grants permissions defined in a Role (or ClusterRole) to a user, group, or ServiceAccount.
- Exists within a namespace.
- Connects subjects to Role.

EXAMPLE: RoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods-binding
  namespace: dev
subjects:
- kind: ServiceAccount
  name: app-sa
  namespace: dev
roleRef:
  kind: Role
  name: read-pods
  apiGroup: rbac.authorization.k8s.io
```

HOW ROLEBINDING WORKS

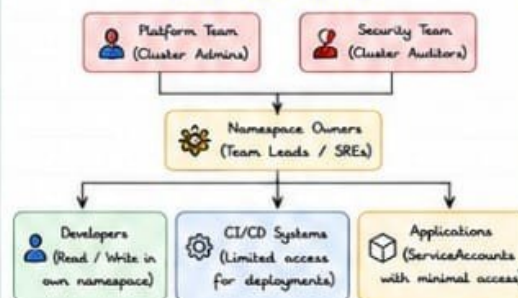


6 PRODUCTION RBAC DESIGN

PRINCIPLES



RECOMMENDED STRUCTURE



EXAMPLE NAMESPACES (dev, staging, prod)

- ✓ Each team owns their namespace
- ✓ No direct access across namespaces
- ✓ Access via RBAC only

7 RBAC QUICK REFERENCE

OBJECT	SCOPE	USED FOR	EXAMPLE
User	Cluster (Global)	Human or external identity	alice, bob, admin
ServiceAccount	Namespace	Workloads / applications	app-sa, cicd-sa
Role	Namespace	Permissions within a namespace	read-pods
ClusterRole	Cluster	Permissions across the cluster	cluster-admin-read
RoleBinding	Namespace	Bind Role to a subject	read-pods-binding
ClusterRoleBinding	Cluster	Bind ClusterRole to a subject	admin-binding

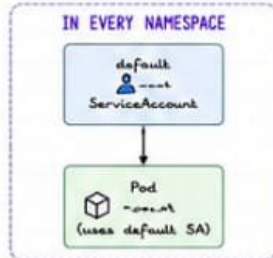
"Good access is invisible to users, but powerful for security."

6. SERVICE ACCOUNTS

"Not for humans. Built for workloads. Identity with purpose."

1. DEFAULT SERVICEACCOUNT

- Kubernetes creates a default ServiceAccount in every namespace.
- Mounted automatically in pods if no other ServiceAccount is specified.
- Has minimal permissions by default (depends on cluster setup).
- Token is auto-mounted unless disabled.



⚡ Never bind cluster-admin to the default ServiceAccount. It is used by many pods by mistake.

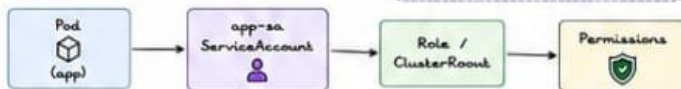
2. CUSTOM SERVICEACCOUNTS

- Create dedicated ServiceAccounts for applications or teams.
- Apply least privilege with RBAC.
- Easier to audit and manage.

EXAMPLE

```

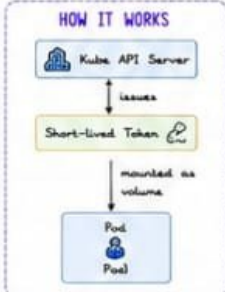
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  namespace: production
  
```



3. TOKEN PROJECTION

- Modern way to provide tokens to pods.
- Short-lived, auto-rotated tokens.
- Mounted via projected volume.
- Improves security over long-lived secrets.

HOW IT WORKS



EXAMPLE: PROJECTED TOKEN

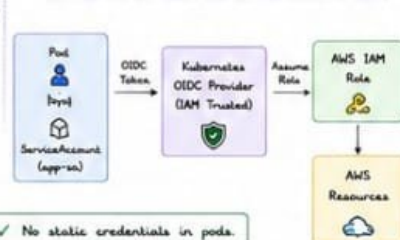
```

apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  serviceAccountName: app-sa
  volumes:
    - name: token
      projected:
        sources:
          - serviceAccountToken:
              path: token
              expirationSeconds: 3600
  containers:
    - name: app
      image: nginx
      volumeMounts:
        - name: token
          mountPath: /var/run/secrets/tokens
  
```

4. WORKLOAD IDENTITY

- Pods authenticate to cloud providers without using static keys.
- Uses OIDC federation.
- Cloud provider trusts Kubernetes ServiceAccount identity.

EXAMPLE: AWS IAM ROLES FOR SERVICE ACCOUNTS (IRSA)



5. BEST PRACTICES

- ✓ Create a dedicated ServiceAccount per application.
- ✓ Follow least privilege (minimal permissions).
- ✓ Disable `automountServiceAccountToken` when not needed.
- ✓ Use token projection (short-lived tokens).
- ✓ Audit ServiceAccount usage regularly.
- ✓ Use Workload Identity for cloud access.
- ✓ Avoid using default ServiceAccount for critical workloads.



POD SECURITY CONTEXT EXAMPLE

```

apiVersion: v1
kind: Pod
metadata:
  name: secure-app
spec:
  serviceAccountName: app-sa
  automountServiceAccountToken: false # disable if not needed
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: app
      image: nginx
  
```

Use `automountServiceAccountToken: false` for pods that don't need to call the Kubernetes API.



6. PRODUCTION SECURITY

AREA	RECOMMENDATION
Least Privilege	Grant only required permissions. Avoid wildcard (*) rules.
No Cluster-Admin	Never bind cluster-admin to any ServiceAccount.
Limit Token Exposure	Do not mount tokens in pods that don't need API access.
Short-Lived Tokens	Use token projection with short expiration.
Audit Regularly	List and review all ServiceAccounts and bindings.
Cloud Access	Use Workload Identity / IRSA / Workload Identity Federation.
Namespace Isolation	Avoid sharing ServiceAccounts across namespaces.

7. COMMON COMMANDS

- `kubectl get serviceaccounts -A` # list SA in all namespaces
- `kubectl describe sa app-sa -n prod` # details
- `kubectl get pods -n prod -o wide` # see SA used by pod
- `kubectl get rolebindings,clusterrolebindings -A` # list bindings
- `kubectl auth can-i get pods --as=system:serviceaccount:prod:app-sa -n prod` # check access

8. QUICK REFERENCE

TOPIC	KEY POINT
Default ServiceAccount	Auto-created in every namespace. Use carefully.
Custom ServiceAccounts	Create dedicated SA for each app or team.
Token Projection	Short-lived, auto-rotated, more secure.
Workload Identity	Authenticate to cloud without static keys.
Best Practice	Least privilege, no cluster-admin, audit, short-lived tokens.

PRO TIP

- Use separate ServiceAccounts for:
 - ✓ Controllers / Operators
 - ✓ Applications
 - ✓ CI/CD Jobs
 - ✓ Monitoring Agents

COMMON MISTAKES

- ✗ Using default ServiceAccount everywhere.
- ✗ Over-privileged permissions.
- ✗ Mounting tokens when not needed.
- ✗ Long-lived manually created token secrets.
- ✗ Sharing ServiceAccounts across teams.

REMEMBER

A ServiceAccount is an identity. RBAC defines what it can do. Security defines how safely it can do it.

"Every workload is an identity. Give it the least. Trust nothing by default."

7. ADMISSION CONTROLLERS

"Every request is checked. Only valid changes enter the cluster."

1 WHAT ARE ADMISSION CONTROLLERS?

- Admission Controllers intercept requests to the Kubernetes API Server before objects are persisted in ETCD.
- They can modify (mutate) or validate (allow/deny) the request.
- They are the first line of defense for security and governance.
- Executed in a defined order.

TWO MAIN TYPES

MUTATING ADMISSION
(Make changes)

VALIDATING ADMISSION
(Validate / Allow or Deny)

Both increase the security, reliability and consistency of the cluster.

2 MUTATING ADMISSION

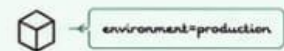
- Can modify the request object before it is persisted.
- Useful for defaulting, injecting sidecars, adding labels, etc.
- Multiple mutating controllers
- Must be idempotent.

EXAMPLES

- ✓ Defaulting resources (limits, requests)
- ✓ Injecting sidecar containers
- ✓ Adding imagePullSecrets
- ✓ Setting tolerations
- ✓ Adding securityContext

EXAMPLE: Add a label to every pod

```
apiVersion: admissionregistration.k8s.io/v1
kind: MutatingAdmissionPolicy (or Webhook)
# Adds label: environment=production
mutation:
  patch: |-
    - op: add
      path: /metadata/labels/environment
      value: production
```



3 VALIDATING ADMISSION

- Does NOT modify the request.
- Validates the request and either allows or denies it.
- Used for enforcing policies, rules, and compliance.
- If any validating controller denies, the request is rejected.

EXAMPLES

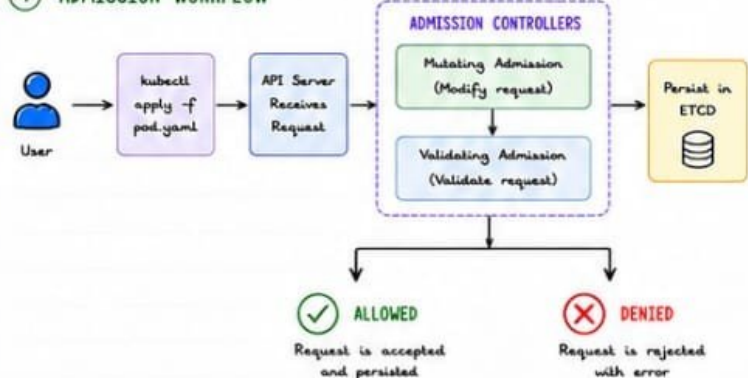
- ✓ Enforce resource limits
- ✓ Deny privileged containers
- ✓ Restrict hostPath volumes
- ✓ Enforce naming standards
- ✓ Block images from untrusted registries

EXAMPLE: Deny privileged containers

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy (or Webhook)
# Deny pods running privileged containers
validation:
  expression: "object.spec.containers.all(c, !c.securityContext.privileged)"
  message: "Privileged containers are not allowed."
```

✗ DENY

4 ADMISSION WORKFLOW



5 POLICY ENFORCEMENT

- Admission controllers help enforce organizational policies.
- Can be implemented using:
 - Built-in controllers
 - Validating/Mutating Webhooks
 - Policy engines (OPA Gatekeeper, Kyverno)



Built-in Controllers



Kyverno



Open Policy Agent (OPA) Gatekeeper

POLICY AREAS

- Security
- Compliance
- Governance
- Cost Control
- Multi-tenancy

6 SECURITY CONTROLS (COMMON USES)

CONTROL	DESCRIPTION	EXAMPLE
Restrict Privileged Containers	Block pods that run with privileged=true	Deny escalation of privileges
Restrict HostPath Volumes	Prevent access to host filesystem	Block sensitive host paths
Image Allow/Deny List	Allow only trusted container images	Deny images from public/untrusted sources
Enforce Resource Limits	Ensure every pod has limits/requests	Prevent noisy-neighbor and DoS
Enforce Labels	Require labels for ownership, env, etc.	Cost tracking, ownership, compliance

7 PRODUCTION EXAMPLES

A. SIDE CAR INJECTION

Automatically inject a logging sidecar into all pods.



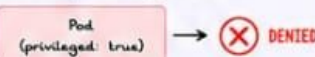
B. ENFORCE RESOURCE LIMITS

Deny pods that do not set resources (CPU/MEM).



C. BLOCK PRIVILEGED PODS

Deny pods with privileged containers or dangerous capabilities.



D. ADD SECURITY CONTEXT

Automatically set securityContext (runAsNonRoot, readOnlyRootFilesystem).



BEST PRACTICES

- ✓ Enable only what you need.
- ✓ Keep webhook timeouts short.
- ✓ Version your webhooks.
- ✓ Use fail-closed for security policies.
- ✓ Monitor and log admission events.

COMMON MISTAKES

- ✗ Disabling admission controllers blindly.
- ✗ Slow webhooks causing API timeouts.
- ✗ Policies that block legitimate workloads.
- ✗ Not testing policies in staging first.

USEFUL COMMANDS

```
$ kubectl get mutatingwebhookconfigurations
$ kubectl get validatingwebhookconfigurations
$ kubectl describe mutatingwebhookconfiguration <name>
$ kubectl auth can-i create pods --as system:serviceaccount:ns:sa
```

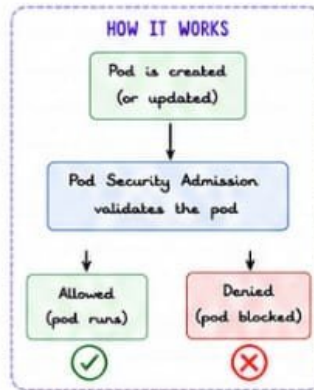
"Admission Controllers are the gatekeepers of your cluster."

8. POD SECURITY

“Lock down the pod. Protect the workload. Secure the cluster.”

1 WHAT IS POD SECURITY?

- Pod Security helps enforce security best practices by restricting how pods are configured.
- It reduces the risk of privilege escalation, container breakout, and host compromise.
- Implemented via Pod Security Standards (PSS) and enforced by the Kubernetes admission system.



2 POD SECURITY STANDARDS (PSS)

LEVEL	DESCRIPTION	USE WHEN
 PRIVILEGED	- No restrictions. - Unrestricted policy. - Equivalent to no policy.	Legacy / trusted systems only. Not recommended.
 BASELINE	- Minimally restrictive. - Blocks known privilege escalation. - Prevents most common risks.	General workloads with basic security needs.
 RESTRICTED	- Heavily restricted. - Follows security best practices. - Strongest level.	High security workloads & production systems.

3 PSS LEVELS EXPLAINED

PRIVILEGED	BASELINE	RESTRICTED
 - Runs as root - Allows privileged containers - Host namespaces allowed - HostPath volumes allowed - No restrictions	 - Disallows privileged=true - Disallows hostPID, hostIPC - Disallows hostNetwork - Restricts capabilities - Allows some hostPath (read-only)	 - Runs as non-root - Disallows privilege escalation - Disallows host namespaces - Disallows hostPath - Drops all capabilities
RISK: VERY HIGH	RISK: MEDIUM	RISK: LOW

4 PSS COMPARISON AT A GLANCE

CONTROL	PRIVILEGED	BASELINE	RESTRICTED
 Run as Non-root	✗ Not required	⚠ Optional	✓ Required
 Privilege Escalation	✗ Allowed	✗ Not allowed	✗ Not allowed
 Privileged Container	✓ Allowed	✗ Not allowed	✗ Not allowed
 Host PID / IPC	✓ Allowed	✗ Not allowed	✗ Not allowed
 Host Network	✓ Allowed	✗ Not allowed	✗ Not allowed
 HostPath Volumes	✓ Allowed	⚠ Read-only allowed	✗ Not allowed
 Capabilities	✓ All allowed	⚠ Limited	✗ Dropped ALL
 AppArmor / Seccomp	⚠ Optional	✓ Required	✓ Required

5 SECURITY ADMISSION (ENFORCEMENT)

- The Pod Security Admission controller enforces PSS based on namespace labels.
- Modes:
 - enforce:** blocks violating pods
 - audit:** logs violations (does not block)
 - warn:** warns users (does not block)

NAMESPACE LABELS

```

# Enforce Restricted
pod-security.kubernetes.io/enforce: restricted
pod-security.kubernetes.io/enforce-version: v1.28

# Audit
pod-security.kubernetes.io/audit: restricted
pod-security.kubernetes.io/audit-version: v1.28

# Warn
pod-security.kubernetes.io/warn: restricted
pod-security.kubernetes.io/warn-version: v1.28
  
```

6 PRODUCTION RECOMMENDATIONS

- Use **RESTRICTED** for all new namespaces. Strongest security, minimal risk.
- Start with **WARN** mode, move to **AUDIT**, then **ENFORCE**. Smooth adoption without breaking workloads.
- Apply namespace-level labels consistently. Security is a team effort.
- Regularly review & update policies. Evolve with new threats and Kubernetes versions.
- Monitor & audit violations. Use logs/metrics to improve posture.

7 EXAMPLE: PODS BLOCKED BY RESTRICTED

Runs as Root ✗ securityContext: runAsUser: 0 ✗ BLOCKED	Privileged Container ✗ securityContext: privileged: true ✗ BLOCKED	HostPath Volume ✗ volumes: - name: data hostPath: path: /data ✗ BLOCKED	Host Network ✗ hostNetwork: true ✗ BLOCKED
---	---	--	--

8 BEST PRACTICES CHECKLIST

- ✓ Enforce the **RESTRICTED** profile in production.
- ✓ Run containers as non-root.
- ✓ Drop all unnecessary capabilities.
- ✓ Use read-only root filesystem.
- ✓ Disallow privilege escalation.
- ✓ Limit hostPath usage (or disallow completely).
- ✓ Keep Kubernetes & policies up to date.
- ✓ Audit and monitor policy violations continuously.



REMEMBER

Pod Security is not about blocking developers. It's about building a secure foundation so your applications can run safely.



COMMON MISTAKES

- ✗ Running everything as root.
- ✗ Using privileged containers unnecessarily.
- ✗ Mounting hostPath without review.
- ✗ Ignoring security warnings and audit logs.



USEFUL COMMANDS

```

$ kubectl get ns --show-labels
$ kubectl label ns <ns> \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/enforce-version=v1.28
$ kubectl describe pod <pod>
$ kubectl logs -n kube-system -l k8s-app=pod-security-webhook
  
```



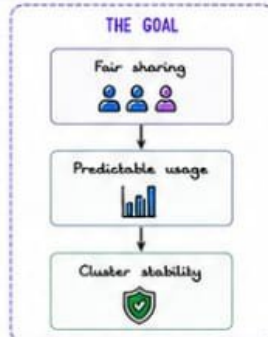
“Secure the pod. Secure the workload. Secure everything.”

9. RESOURCE QUOTAS & LIMITRANGES

"Manage resources today. Avoid outages tomorrow."

1 WHY IT MATTERS

- Kubernetes resources are finite.
- Without constraints, one team can starve others.
- Resource Quotas & LimitRanges ensure fairness, stability, and governance.
- They are the foundation of multi-tenant clusters.



2 RESOURCEQUOTA

- Limits the total amount of resources that can be used in a namespace.
- Applies to compute, storage, objects, and more.
- Prevents "noisy neighbors" from consuming everything.

EXAMPLE: ResourceQuota

```

apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-a-quota
  namespace: team-a
spec:
  hard:
    requests.cpu: "10"           # total CPU requests
    requests.memory: "20Gi"      # total memory requests
    limits.cpu: "20"             # total CPU limits
    limits.memory: "40Gi"        # total memory limits
    pods: "50"                   # max pods
    persistentvolumeclaims: "10" # max PVCs
    services.loadbalancers: "2"  # max LB services
  
```

3 LIMITRANGE

- Sets default requests/limits for containers.
- Ensures containers have same resource boundaries.
- Prevent misconfigured pods from consuming too much or too little.

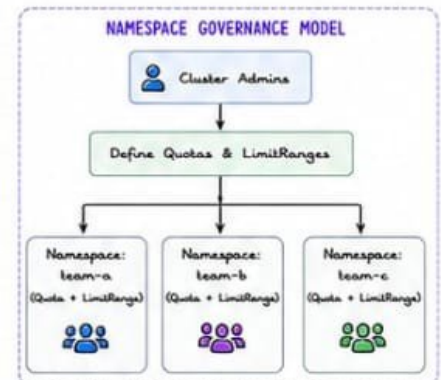
EXAMPLE: LimitRange

```

apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: team-a
spec:
  limits:
    - type: Container
      default:
        cpu: "500m"
        memory: "512Mi"
      defaultRequest:
        cpu: "250m"
        memory: "256Mi"
      max:
        cpu: "2"
        memory: "2Gi"
      min:
        cpu: "100m"
        memory: "128Mi"
  
```

4 NAMESPACE GOVERNANCE

- Apply quotas & limits per namespace to isolate teams.
- Gives teams freedom to deploy without affecting others.
- Enables clear boundaries and accountability.



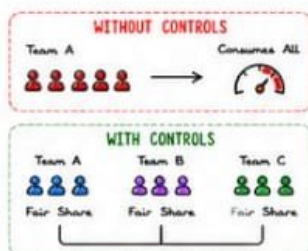
6 CAPACITY PLANNING

- Use quotas to plan cluster capacity per team/workload.
- Leave headroom for system components and future growth.
- Review usage regularly and adjust quotas.



5 FAIR RESOURCE SHARING

- Quotas prevent one team from taking all available resources.
- LimitRanges ensure pods are configured responsibly.
- Everyone gets a fair share of the cluster.



7 PRODUCTION EXAMPLES

SCENARIO	RESOURCEQUOTA (EXAMPLE)	LIMITRANGE (EXAMPLE)	BENEFIT
 Dev Team Namespace	• requests.cpu: "5" • requests.memory: "10Gi" • pods: "30"	• defaultRequest: cpu=200m, memory=256Mi • default: cpu=500m, memory=512Mi • max: cpu=1, memory=1Gi	Prevents dev workloads from overconsuming.
 Data Team Namespace	• requests.cpu: "20" • requests.memory: "50Gi" • persistentvolumeclaims: "10"	• defaultRequest: cpu=500m, memory=1Gi • max: cpu=4, memory=4Gi	Ensures heavy jobs don't impact other teams.
 Shared Platform Namespace	• services.loadbalancers: "2" • configmaps: "50"	• defaultRequest: cpu=100m, memory=128Mi • max: cpu=500m, memory=256Mi	Protects platform services from abuse.

BEST PRACTICES

- ✓ Apply both ResourceQuota & LimitRange.
- ✓ Start restrictive, then relax as needed.
- ✓ Use meaningful defaults.
- ✓ Monitor usage and enforce limits.
- ✓ Document and review regularly.

COMMON MISTAKES

- ✗ No quotas or limits (chaos!).
- ✗ Setting limits without requests.
- ✗ Very high defaults.
- ✗ Forgetting to review & adjust.

USEFUL COMMANDS

```

$ kubectl get resourcequota -n <namespace>
$ kubectl get limitrange -n <namespace>
$ kubectl describe resourcequota <name> -n <ns>
$ kubectl describe limitrange <name> -n <ns>
$ kubectl top namespace
  
```

"Bound your resources. Protect your cluster. Empower your teams."

10. PRIORITY CLASSES & PREEMPTION

"The right workload, at the right time, gets the resources it needs."

1 WHAT IS PRIORITY?

- Priority helps Kubernetes decide which pods get resources first.
- Higher priority pods can preempt lower priority pods.
- Critical workloads stay available even under resource pressure.
- Built using PriorityClass objects.

KEY CONCEPT

Not all workloads are equal.

Priority ensures what matters most runs when it matters most.

2 PRIORITYCLASS

- Defines a set of priorities for pods.
- Each PriorityClass has a value (integer).
- Higher value = higher priority.
- Global scope (not namespace).

EXAMPLE: PRIORITYCLASS

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: critical-app
  value: 1000000 # higher number = higher priority
globalDefault: false
description: "Critical application priority"
preemptionPolicy: PreemptLowerPriority
```

3 POD PRIORITY

- Pods set priority by referencing a PriorityClass.
- If not set, pods use the default priority (0).
- Higher priority pods are scheduled first.

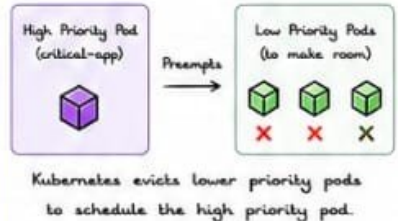
EXAMPLE: POD USING PRIORITYCLASS

```
apiVersion: v1
kind: Pod
metadata:
  name: critical-pod
spec:
  priorityClassName: critical-app
  containers:
    - name: app
      image: myapp:v1
```

4 PREEMPTION

- When no resources are available, Kubernetes may remove lower priority pods to make room.
- This is called preemption.
- Only works if the higher priority pod has a PriorityClass with preemption enabled.

PREEMPTION EXAMPLE



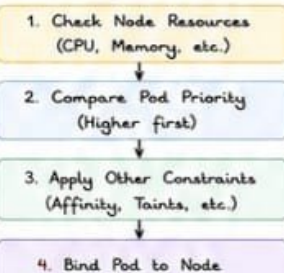
Always create a PriorityClass for important workloads. Never rely only on default (0) priority.

5 SCHEDULING ORDER

- Kubernetes considers these factors when scheduling:

1. Node resources
2. Pod priority
3. Other constraints (affinity, taints, etc.)

SCHEDULING ORDER FLOW



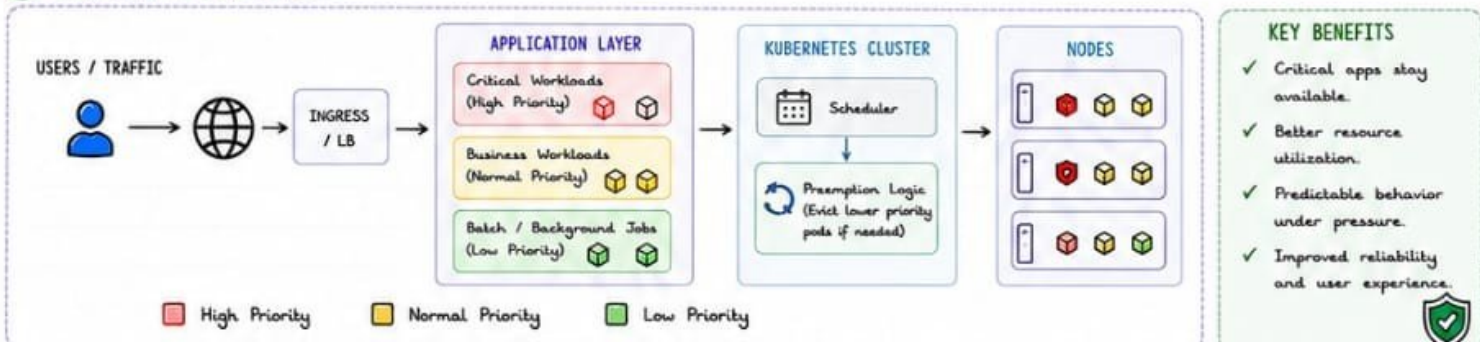
6 CRITICAL WORKLOADS

- Use high priority for workloads that must always run:
 - ✓ Customer-facing apps
 - ✓ Databases
 - ✓ Monitoring & Alerting
 - ✓ Security services
 - ✓ System components (CNI, DNS, etc.)

EXAMPLE PRIORITY TIERS



7 PRODUCTION ARCHITECTURE



BEST PRACTICES

- ✓ Define clear priority tiers.
- ✓ Keep high priority classes for only critical workloads.
- ✓ Test preemption in staging.
- ✓ Monitor evictions and events.
- ✓ Document your priority strategy.

COMMON MISTAKES

- ✗ Giving high priority to too many pods.
- ✗ Not setting a PriorityClass (uses default 0).
- ✗ Using preemption for non-critical workloads.
- ✗ Ignoring eviction events and alerts.

USEFUL COMMANDS

```
$ kubectl get priorityclass
$ kubectl describe priorityclass <name>
$ kubectl get pods -A --sort-by=.spec.priorityClassName
$ kubectl get events --sort-by=.lastTimestamp
$ kubectl top nodes
$ kubectl describe pod <pod-name>
```

"Priority is about impact. Preemption is about protection. Use both wisely."

11. HELM

"Helm makes Kubernetes easy to install, manage, and upgrade."

1 WHAT IS HELM?

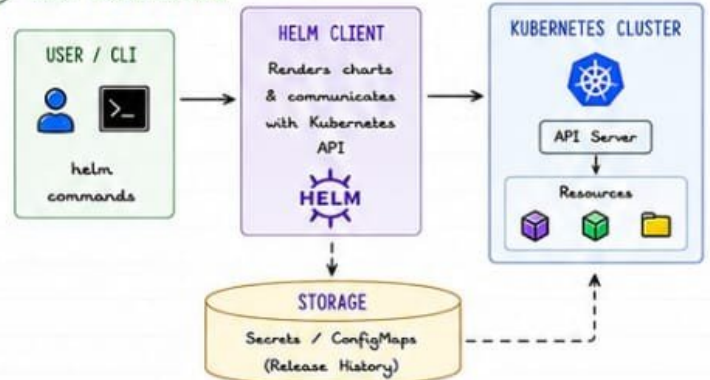
- Helm is the package manager for Kubernetes.
- It helps you define, install, and upgrade even the most complex Kubernetes applications.
- Helm uses Charts to package Kubernetes resources.
- It keeps track of Releases in the cluster.



HELM IN ONE LINE

Helm = Charts + Releases
+ Versioning + Rollbacks

2 HELM ARCHITECTURE



3 HELM CHARTS

- A Helm Chart is a collection of files that describe a related set of Kubernetes resources.
- Charts are versioned, shareable, and reusable.
- Stored in chart repositories (public or private).

CHART STRUCTURE

```

mychart/
├── Chart.yaml      # Chart metadata
├── values.yaml     # Default values
├── charts/         # Dependencies
├── templates/      # KBs manifests
│   ├── deployment.yaml
│   ├── service.yaml
│   └── _helpers.tpl # Template helpers
└── NOTES.txt      # Info after install
  
```

4 TEMPLATES

- Templates are written in Go template language.
- They generate Kubernetes manifests dynamically.
- Make charts flexible and reusable.

TEMPLATE EXAMPLE (deployment.yaml)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "mychart.fullname" . }}
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    metadata:
      labels:
        app: {{ .Chart.Name }}
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: {{ .Values.image.repository }}{{ .Values.image.tag }}
  
```

5 VALUES

- Values are data passed to templates.
- Stored in values.yaml (default) and can be overridden.
- Supports multiple environments.

VALUES EXAMPLE (values.yaml)

```

replicaCount: 2
image:
  repository: nginx
  tag: 1.25
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
resources:
  limits:
    cpu: 500m
    memory: 512Mi
  requests:
    cpu: 100m
    memory: 128Mi
  
```

OVERRIDE VALUES

- Using -f file
helm install app /mychart -f prod-values.yaml
- Using --set
helm install app /mychart --set replicaCount=3 --set image.tag=1.26

6 RELEASES

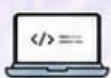
- A Release is a specific instance of a chart running in a cluster.
- Each release has a unique name.
- Helm keeps revision history for rollbacks.

RELEASE LIFECYCLE



7 PRODUCTION WORKFLOW

1. DEVELOP / PACKAGE



Create or modify Helm chart locally.

2. LINT & TEST



helm lint /chart
Test templates & values.

3. PUSH TO REPO



Push chart to Helm repository (e.g. OCI, ChartMuseum)

4. DEPLOY



helm install myapp /chart -n prod -f values-prod.yaml

5. UPGRADE



helm upgrade myapp /chart -n prod -f values-prod.yaml

6. ROLLBACK



helm rollback myapp 2 -n prod

7. MONITOR



Monitor resources, logs & metrics.

COMMON HELM COMMANDS

```

$ helm repo add stable https://charts.helm.sh/stable # Add repo
$ helm repo update # Update repos
$ helm search repo nginx # Search charts
$ helm install myapp /chart -n prod -f values.yaml # Install
$ helm list -n prod # List releases
$ helm upgrade myapp /chart -n prod # Upgrade
$ helm rollback myapp 2 -n prod # Rollback
$ helm uninstall myapp -n prod # Uninstall
$ helm status myapp -n prod # Release status
  
```

BEST PRACTICES

- ✓ Version your charts.
- ✓ Use values files per environment.
- ✓ Keep secrets out of values.yaml.
- ✓ Use --atomic for safe upgrades.
- ✓ Test with --dry-run before install.
- ✓ Store charts in private repository.
- ✓ Monitor release history & rollbacks.

COMMON MISTAKES

- ✗ Hardcoding values in templates.
- ✗ Not using version control for charts.
- ✗ Ignoring lint and template tests.
- ✗ Storing secrets in values.yaml.
- ✗ Skipping rollback on failed upgrades.

"Helm brings order to Kubernetes. Package once, deploy anywhere."

12. KUSTOMIZE

“Native configuration management for Kubernetes.”

① WHAT IS KUSTOMIZE?

- Kustomize is a Kubernetes native configuration management tool.
- It lets you customize YAML files without modifying the original.
- Uses a “base + overlays” approach.
- Built into kubectl (no extra install).
- Great for managing multiple environments (dev, staging, prod).

KEY BENEFITS

- ✓ No templating language to learn
- ✓ Simple and declarative
- ✓ Version controlled configs
- ✓ Environment specific customizations
- ✓ Production ready

② BASES

- Base contains the common, shared Kubernetes manifests.
- Single source of truth.
- Reused by all environments.
- Should be environment agnostic (no hardcoded values).

EXAMPLE: BASE STRUCTURE

```
my-app/
├── base/
│   ├── kustomization.yaml
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── configmap.yaml
│   └── ingress.yaml
└── overlays/
```

Common manifests

③ OVERLAYS

- Overlays contain environment specific customizations.
- They reference the base and apply changes.
- Each environment (dev, staging, prod) has its own overlay.
- Keeps environments isolated and manageable.

EXAMPLE: OVERLAY STRUCTURE

```
my-app/
├── overlays/
│   ├── dev/
│   │   ├── kustomization.yaml
│   │   └── patches/
│   ├── staging/
│   │   ├── kustomization.yaml
│   │   └── patches/
│   └── prod/
│       ├── kustomization.yaml
│       └── patches/
└── base/
```

Dev
Staging
Prod

④ PATCHES

- Patches modify specific fields of resources.
- Apply only the changes you need.
- Types:
 - Strategic Merge Patch
 - JSON 6902 Patch
 - Patches (inline)
- Keeps changes small, clean and focused.

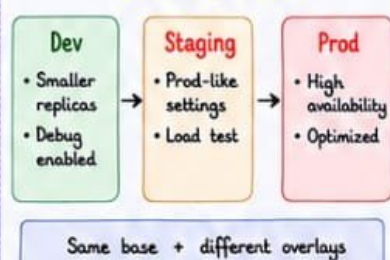
EXAMPLE: STRATEGIC MERGE PATCH

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3 # override replicas
  template:
    spec:
      containers:
        - name: app
          image: my-app:v2 # new image
```

⑤ ENVIRONMENT MANAGEMENT

- Manage multiple environments using overlays.
- Promote the same code from dev → staging → prod.
- Keeps configuration consistent and traceable.
- Works great with GitOps.

ENVIRONMENT FLOW



⑥ PRODUCTION STRUCTURE

RECOMMENDED KUSTOMIZE LAYOUT

```
my-app/
├── base/
│   ├── kustomization.yaml
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── configmap.yaml
│   └── patches/
├── overlays/
│   ├── dev/
│   │   ├── kustomization.yaml
│   │   └── patches/
│   ├── staging/
│   │   ├── kustomization.yaml
│   │   └── patches/
│   └── prod/
│       ├── kustomization.yaml
│       └── patches/
└── patches/
```

WHY THIS WORKS

- ✓ Separation of concerns
- ✓ Reuses common manifests
- ✓ Environment specific overrides
- ✓ Easy to maintain
- ✓ GitOps friendly
- ✓ Scales for large projects

⑦ BEST PRACTICES

- ✓ Keep the base generic (no environment specific values).
- ✓ Use overlays for all customizations.
- ✓ Prefer small, focused patches.
- ✓ Use labels and annotations for tracking.
- ✓ Validate changes with: `kubectl kustomize`.
- ✓ Commit and review changes (GitOps ready).
- ✓ Keep directory structure consistent across projects.
- ✓ Document your Kustomize setup.



COMMON MISTAKES

- ✗ Modifying base files directly.
- ✗ Hardcoding environment values in base.
- ✗ Using huge patches instead of small ones.
- ✗ Mixing unrelated changes in one patch.
- ✗ Forgetting to validate output.
- ✗ Poor directory structure.



USEFUL COMMANDS

```
$ kubectl kustomize . # Render manifests
$ kubectl apply -k overlays/dev # Deploy to dev
$ kubectl apply -k overlays/prod # Deploy to prod
$ kubectl diff -k overlays/staging # Preview changes
$ kubectl kustomize . | less # View output
```



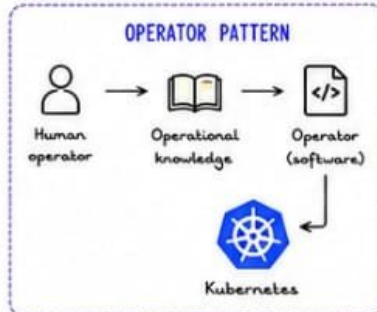
“One base. Many overlays. Infinite environments.”

13. OPERATORS

“Operators turn Kubernetes into an automation engine.”

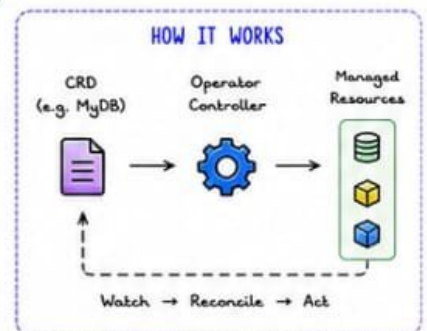
① WHAT IS AN OPERATOR?

- An Operator is an application that extends Kubernetes API using Custom Resources.
- It encodes human operational knowledge into software.
- It automates day-2 operations for complex stateful applications.
- Built on the Operator Pattern.



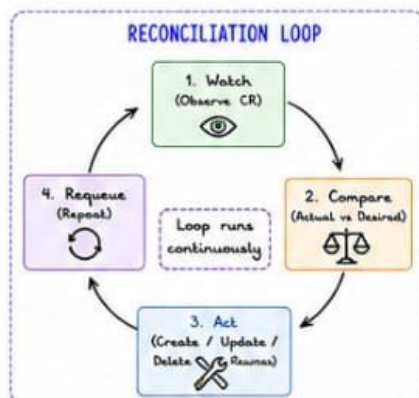
② CUSTOM CONTROLLERS

- Operators use Custom Resources (CRDs) to represent application instances.
- The Operator watches these resources and takes actions.
- It is a controller running inside the cluster.



③ RECONCILIATION LOOP

- Core of every Operator.
- Continuously compares the desired state (from CR) with the current state.
- Takes actions to move the current state toward the desired state.



④ AUTOMATION

- Operators automate:
 - ✓ Installation
 - ✓ Upgrades
 - ✓ Backups
 - ✓ Failover
 - ✓ Scaling
 - ✓ Configuration
 - ✓ Patching
 - ✓ Monitoring
- Reduces manual work and human errors.

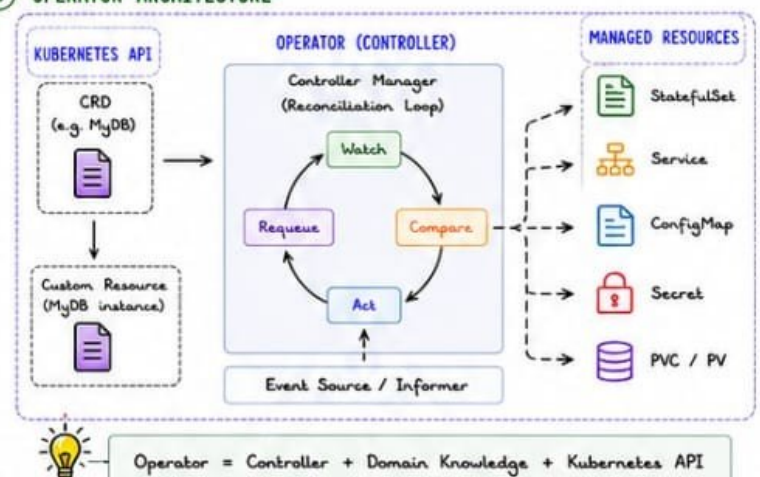


⑤ PRODUCTION USE CASES

- Operators shine for complex, stateful, mission-critical systems.
- Examples of popular Operator use in production.



⑥ OPERATOR ARCHITECTURE



BEST PRACTICES

- ✓ Use Operators for complex, stateful applications.
- ✓ Make CRDs simple and user-friendly.
- ✓ Implement idempotent reconciliation logic.
- ✓ Handle failures and edge cases gracefully.
- ✓ Add proper logging, metrics and alerts.
- ✓ Test upgrades in staging before production.



COMMON MISTAKES

- ✗ Building Operators for simple stateless apps.
- ✗ Long reconciliation loops (slow responses).
- ✗ Not handling errors and retries.
- ✗ Overusing finalizers.
- ✗ Missing RBAC permissions.



USEFUL COMMANDS

```

$ kubectl get crd
$ kubectl get <crd> -A
$ kubectl describe <crd> <name>
$ kubectl logs deploy/<operator-name> -n <ns>
$ kubectl get events -n <ns> --sort-by=.lastTimestamp
$ kubectl explain <crd>.spec
  
```



“Operators encode expertise once, and run it forever.”

14. CUSTOM RESOURCE DEFINITIONS (CRDs)

“CRDs let you extend Kubernetes with your own APIs.”

1 WHAT ARE CRDs?

- CRDs (Custom Resource Definitions) extend the Kubernetes API.
- They allow you to create your own Custom Resources.
- Once a CRD is installed, Kubernetes API server can store and serve instances of that custom resource.
- CRDs are cluster-scoped (definitions), while Custom Resources are namespaced (instances).

EXAMPLE

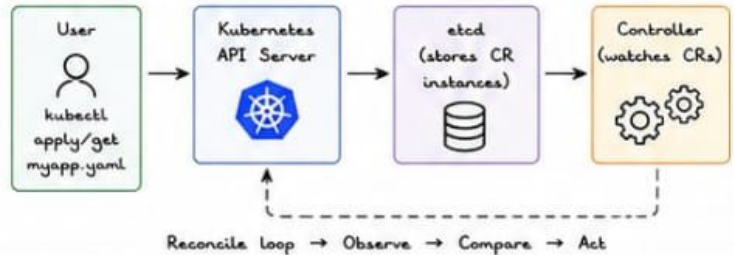
Built-in:

- pods
- deployments
- services

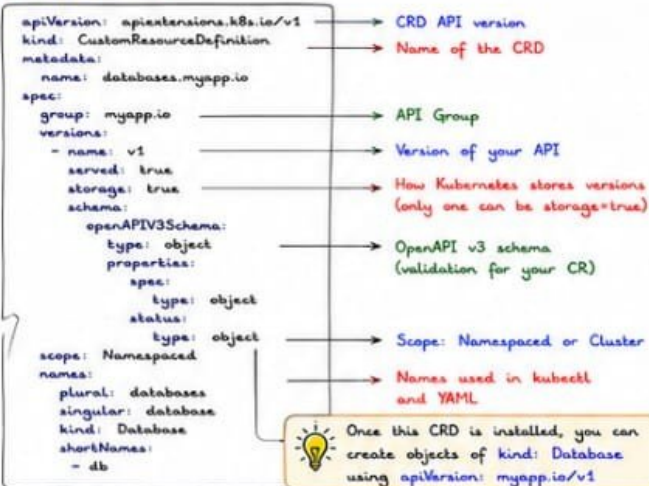
Custom (via CRD):

- databases.myapp.io
- backups.myapp.io
- reports.myapp.io

2 HOW CRDs WORK



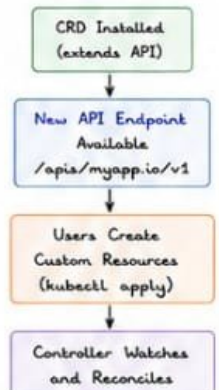
3 CRD ANATOMY



4 CUSTOM APIS & API EXTENSIONS

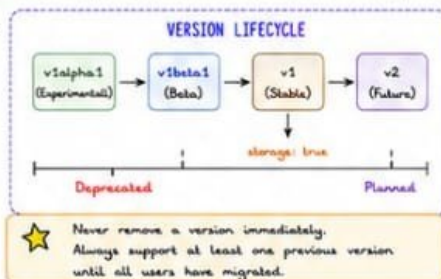
- CRDs add new API endpoints to Kubernetes.
- Example:
`GET /apis/myapp.io/v1/namespaces/dev/databases`
- You can define multiple versions (v1alpha1, v1beta1, v1).
- Kubernetes handles storage, conversion, and validation.
- Built-in discovery, auth, RBAC, and admission apply automatically.

API EXTENSION FLOW



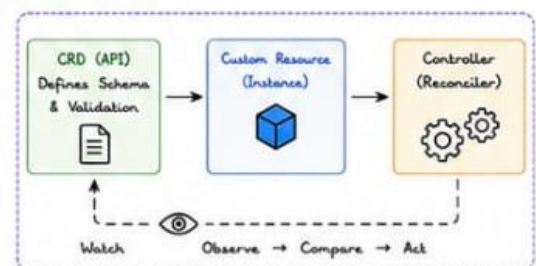
5 VERSIONING STRATEGY

- Support multiple versions of your API.
- Mark one version as storage version.
- Use conversion webhook for complex migrations.
- Deprecate old versions gracefully.



6 CRD ↔ CONTROLLER RELATIONSHIP

- CRD defines the API (what).
- Controller implements the behavior (how).
- Controller watches CR instances and keeps the desired state.
- This is the heart of the Operator pattern.



7 PRODUCTION EXAMPLES

OPERATOR / PROJECT	CRDs PROVIDED	PURPOSE
 PostgreSQL Operator	PostgreSQL, Cluster	Manage PostgreSQL clusters
 MySQL Operator	MySQL, Backup	Manage MySQL HA, backups
 MongoDB Operator	MongoDBCommunity, Backup	MongoDB clusters & backups
 Redis Operator	Redis, RedisCluster	Manage Redis & failover
 Strimzi (Kafka Operator)	Kafka, KafkaConnect	Run & manage Kafka on K8s
 cert-manager	Certificate, Issuer, ClusterIssuer	Manage TLS certificates

8 BEST PRACTICES

- ✓ Design CRDs with clear, future-proof schemas.
- ✓ Use validation (OpenAPI v3) to reject bad config.
- ✓ Follow Kubernetes API conventions (naming, status, conditions).
- ✓ Keep controllers idempotent and resilient.
- ✓ Implement proper status & conditions for observability.
- ✓ Document your API for users.
- ✓ Use conversion webhooks for version upgrades.

COMMON MISTAKES

- ✗ Breaking changes without versioning.
- ✗ Overly complex CRDs (too many responsibilities).
- ✗ Not providing status or conditions.
- ✗ Ignoring RBAC and security.

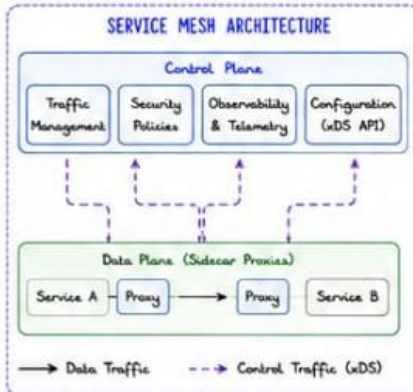
“CRDs extend the API. Operators bring it to life.”

15. SERVICE MESH

"A Service Mesh makes microservices **reliable**, **secure** and **observable**."

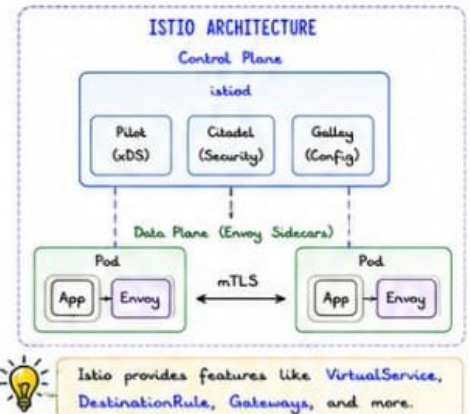
1 SERVICE MESH OVERVIEW

- A Service Mesh is a dedicated infrastructure layer for handling service-to-service communication.
- It runs as a sidecar proxy with each service.
- The data plane handles traffic, while the control plane manages and configures the mesh.
- Benefits: traffic control, security, observability, reliability.



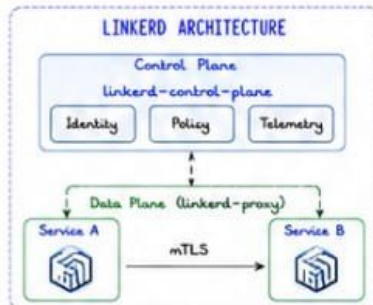
2 ISTIO

- Istio is the most popular service mesh on Kubernetes.
- Provides powerful traffic management, security and observability.
- Uses Envoy proxy as the data plane.
- Control plane components: istiod, pilot, citadel, galley.



3 LINKERD

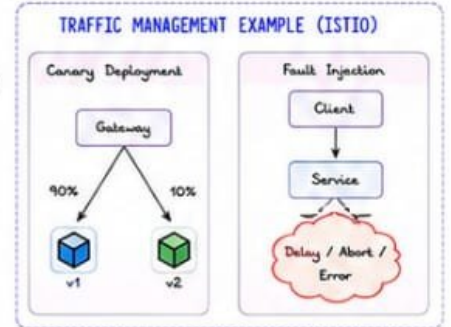
- Linkerd is a lightweight service mesh.
- Easy to install and operate.
- Uses Rust-based proxy (linkerd-proxy).
- Focus: reliability, security, and observability with simplicity.



Linkerd is CNCF graduated and designed for production simplicity.

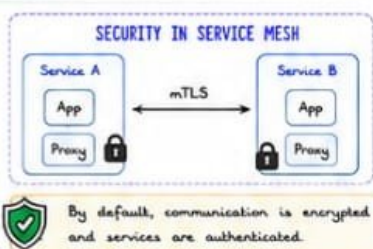
4 TRAFFIC MANAGEMENT

- Control how traffic flows between services.
- ✓ Routing (weights, rules)
- ✓ Retries
- ✓ Timeouts
- ✓ Circuit Breaking
- ✓ Fault Injection
- ✓ Canary / Blue-Green
- ✓ Traffic Mirroring



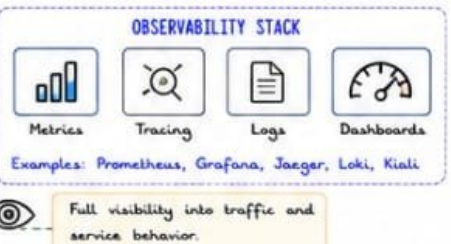
5 SECURITY

- ✓ mTLS between all services
- ✓ Identity-based access control
- ✓ Authorization policies
- ✓ Certificate management
- ✓ Zero-trust communication



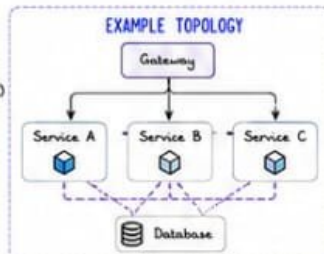
6 OBSERVABILITY

- ✓ Metrics (request rate, latency, errors)
- ✓ Distributed tracing
- ✓ Logs collection
- ✓ Service topology
- ✓ Health checks



7 PRODUCTION USE CASES

- Microservices communication security (mTLS)
- Zero-downtime deployments (canary, blue-green)
- Traffic shaping and routing
- Observability in complex architectures
- Compliance and audit requirements
- Resilience (retries, circuit breaking, timeouts)



8 BEST PRACTICES

- ✓ Start simple. Enable mesh features gradually.
- ✓ Use meaningful service identities and namespaces.
- ✓ Enforce mTLS and authorization policies.
- ✓ Monitor the mesh (control plane & data plane).
- ✓ Keep proxies and control plane updated.
- ✓ Test traffic policies in staging before production.

KEY SERVICE MESH FEATURES

- ★ Service discovery
- ★ Traffic management
- ★ Security (mTLS, authZ)
- ★ Observability
- ★ Reliability

COMMON MISTAKES

- ✗ Enabling all features at once.
- ✗ Ignoring proxy resource usage.
- ✗ Missing mTLS or weak policies.
- ✗ No observability for the mesh.
- ✗ Not testing traffic rules.

USEFUL COMMANDS (ISTIO)

```
$ istioctl install
$ istioctl proxy-status
$ istioctl proxy-config routes <pod>
$ istioctl analyze
$ kubectl get gateways,virtualservices,destinationrules
```

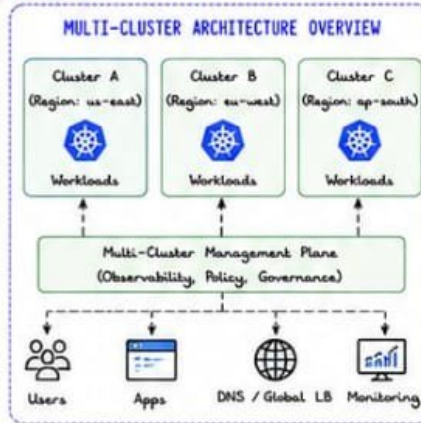
"Service Mesh is the **network layer** your **microservices** deserve."

16. MULTI-CLUSTER KUBERNETES

"Run anywhere. Connect everywhere. Recover from anything."

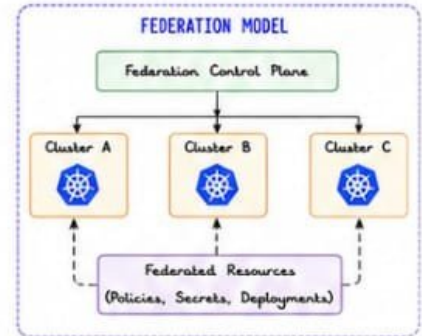
1 MULTI-CLUSTER ARCHITECTURE

- Multiple Kubernetes clusters work together as a single logical platform.
- Improves resilience, scalability and latency.
- Workloads can run across regions, clouds or datacenters.
- Each cluster remains independent but connected.



2 FEDERATION CONCEPTS

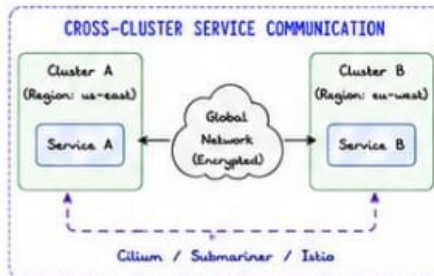
- Federation connects multiple clusters for shared control.
- Resources can be synchronized across clusters.
- Policies and identities can be enforced globally.
- Tools: Kubernetes Federation (v1), Kubefed, Fleet, Rancher Fleet, Cluster API.



Federation = Manage many clusters as a single unit.

3 CROSS-CLUSTER NETWORKING

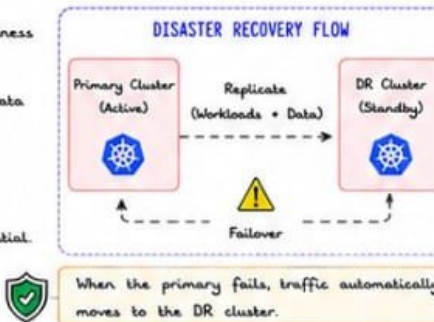
- Enables communication between services in different clusters.
- Ensures secure, fast and reliable connectivity.
- Solutions: Cilium Cluster Mesh, Submariner, Istio Multi-Cluster.



Cross-cluster networking makes services talk like they are in the same cluster.

5 DISASTER RECOVERY

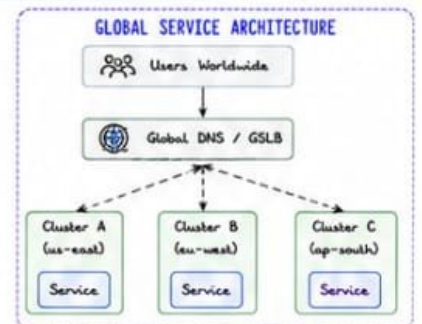
- Multi-cluster improves business continuity.
- Replicate workloads and data across clusters.
- Failover traffic to healthy clusters during outages.
- Regular DR drills are essential.



When the primary fails, traffic automatically moves to the DR cluster.

4 GLOBAL SERVICES

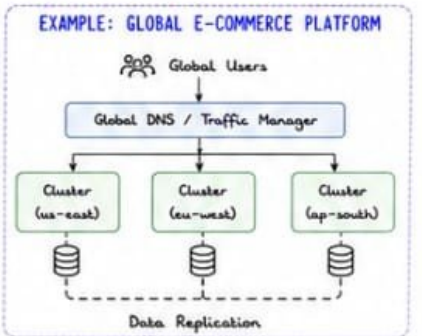
- Expose services across multiple clusters to users worldwide.
- Use Global Load Balancers and DNS for traffic routing.
- Examples: AWS Global Accelerator, Cloudflare, Google Cloud Global LB.



Global DNS routes users to the nearest healthy cluster.

6 PRODUCTION USE CASES

- High Availability
- Low Latency
- Data Sovereignty
- Cloud Bursting
- Blue/Green Across Regions
- Disaster Recovery



7 KEY BENEFITS

- ✓ High availability and resilience
- ✓ Better performance (run closer to users)
- ✓ Scalability across regions and clouds
- ✓ Isolation and security between environments
- ✓ Disaster recovery and business continuity
- ✓ Cost optimization across providers



8 BEST PRACTICES

- ✓ Keep clusters loosely coupled.
- ✓ Use global observability (logging, metrics, tracing).
- ✓ Standardize networking and security policies.
- ✓ Automate cluster provisioning (Cluster API).
- ✓ Test failover and DR regularly.
- ✓ Document runbooks for cross-cluster operations.



REMEMBER

Multi-cluster is not just running more clusters. It's about connecting them with purpose.



COMMON MISTAKES

- ✗ Tight coupling between clusters.
- ✗ Ignoring network latency.
- ✗ No global monitoring and alerting.
- ✗ Unplanned failover (no DR test).
- ✗ Inconsistent policies and versions.



TOP TOOLS

- Cilium Cluster Mesh
- Istio Multi-Cluster
- Rancher Fleet
- Submariner
- Kubefed / Fleet
- Cluster API



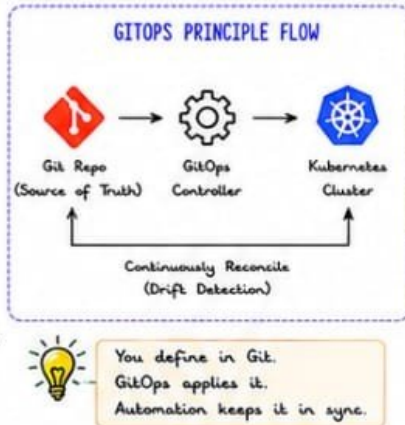
"One cluster can fail. Many clusters keep you in the game."

17. GITOPS

"Git is the source of truth. Everything else is automation."

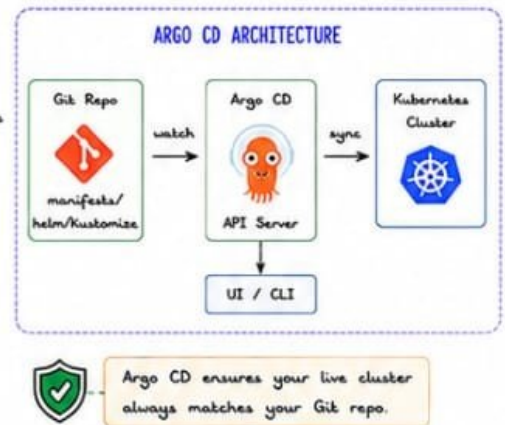
1 GITOPS PRINCIPLES

- Git is the single source of truth.
- Declarative: desired state is stored in Git.
- Automated: changes are applied automatically.
- Versioned, auditable and reversible.
- Continuous reconciliation ensures the cluster matches Git.



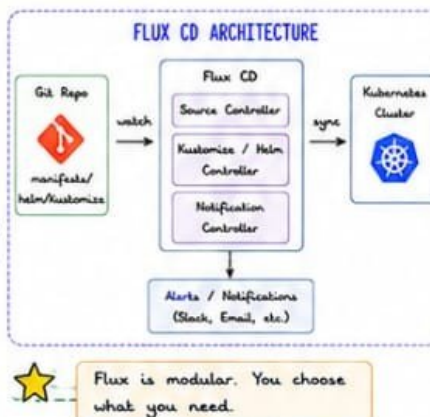
2 ARGO CD

- Declarative, GitOps CD tool for Kubernetes.
- Monitors Git repositories and applies changes.
- Web UI for visibility and management.
- Supports multi-cluster deployments.



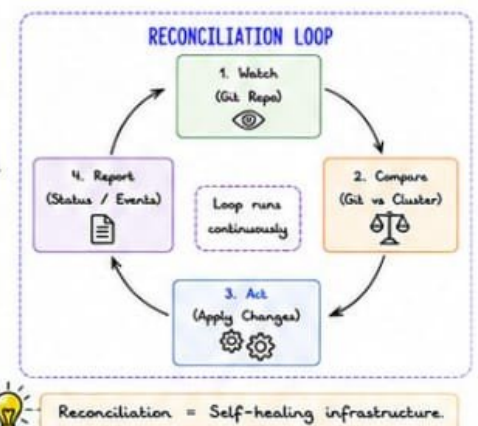
3 FLUX CD

- CNCF graduated GitOps toolkit.
- Built for Kubernetes and native to the ecosystem.
- Uses controllers (Kustomize Controller, Helm Controller, etc.)
- Highly customisable and extensible.



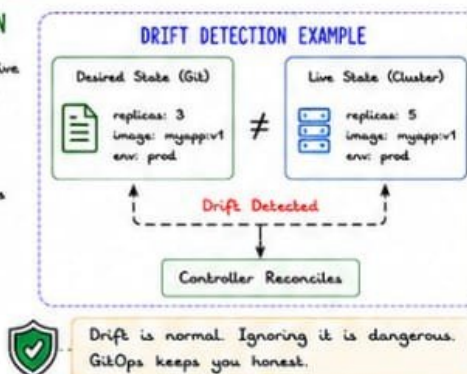
4 RECONCILIATION

- The GitOps controller continuously compares desired state (Git) with live state (cluster).
- If drift is detected, it automatically reconciles.
- Ensures self-healing and consistency.



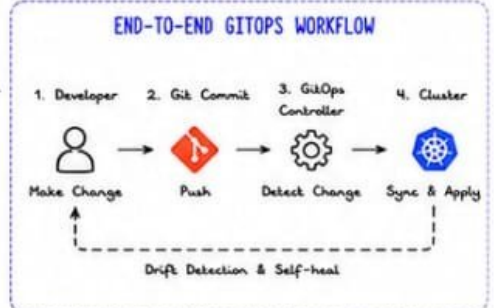
5 DRIFT DETECTION

- Drift happens when live cluster state differs from Git.
- Could be accidental changes, manual edits or failures.
- GitOps detects and corrects the drift automatically.



6 PRODUCTION WORKFLOW

- Developers change code or manifests.
- Commit and push to Git.
- GitOps controller detects changes.
- Controller syncs and applies to clusters.
- Rollback by reverting Git commit.



GITOPS BENEFITS

- ✓ Single source of truth
- ✓ Automated deployments
- ✓ Audit trail and version history
- ✓ Rollback and recovery made easy
- ✓ Consistency across environments
- ✓ Self-healing and drift correction

COMMON PITFALLS

- ✗ Manual changes in cluster (causes drift)
- ✗ Ignoring alerts and sync failures
- ✗ Storing secrets in plain text in Git
- ✗ Missing RBAC for GitOps components
- ✗ Not testing changes before pushing

USEFUL COMMANDS



```
$ argocd app list
$ argocd app sync <app-name>
$ argocd app get <app-name>
$ argocd app diff <app-name>
$ flux get all
$ flux reconcile kustomization <name>
```

"Git stores the intent. GitOps delivers the state."

18. PLATFORM ENGINEERING

"Platform Engineering builds the paved roads so developers can move fast."

1 INTERNAL DEVELOPER PLATFORMS (IDP)

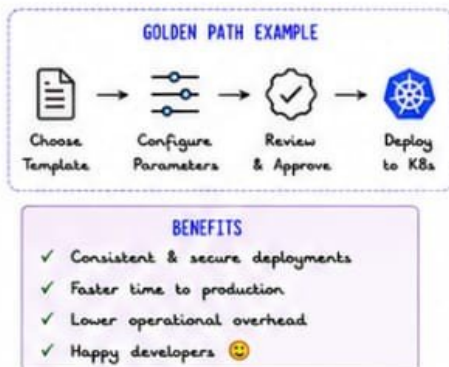
- An Internal Developer Platform is a set of tools, services and workflows that abstract complexity.
- Provides everything developers need to build, test and deploy.
- Improves developer experience and productivity.



IDP = The backstage that makes the developer experience amazing.

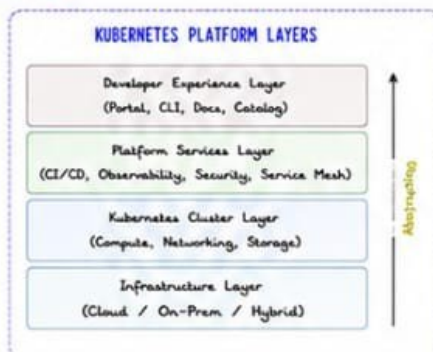
3 GOLDEN PATHS

- Golden paths are pre-approved, secure and optimized ways to build and deploy.
- They reduce cognitive load and prevent mistakes.
- Balance between flexibility and standards.



5 KUBERNETES AS A PLATFORM

- Kubernetes is the foundation of modern platforms.
- Provides portability, scalability and resilience.
- Everything runs on Kubernetes: apps, databases, queues, etc.
- Expose K8s via simple abstractions.



KEY BENEFITS

- ✓ Faster developer onboarding
- ✓ Consistent, secure and compliant deployments
- ✓ Self-service reduces operational bottlenecks
- ✓ Higher productivity and developer satisfaction
- ✓ Standardization without sacrificing agility



COMMON PITFALLS

- ✗ Over-engineering the platform
- ✗ Ignoring developer experience
- ✗ Too many tools, no integration
- ✗ Weak documentation and support
- ✗ No clear ownership or roadmap



USEFUL COMMANDS (K8S)

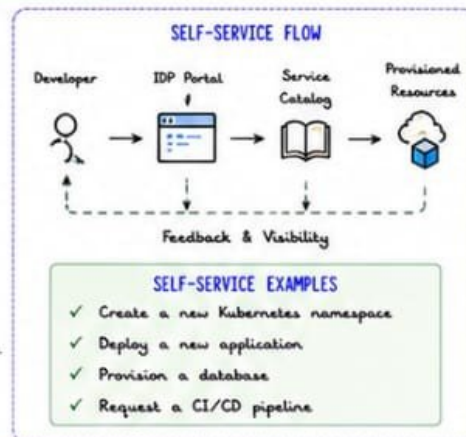


```

$ kubectl get pods -A
$ kubectl get svc -A
$ kubectl get namespaces
$ kubectl describe pod <pod-name>
$ kubectl top nodes
$ kubectl get events --sort-by=.lastTimestamp
  
```

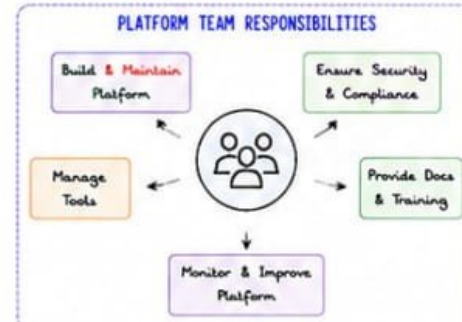
2 SELF-SERVICE INFRASTRUCTURE

- Developers can provision what they need, when they need it.
- No more waiting on operations tickets.
- Guardrails ensure security, cost and compliance.
- Automation + policies make it safe and reliable.



4 PLATFORM TEAMS

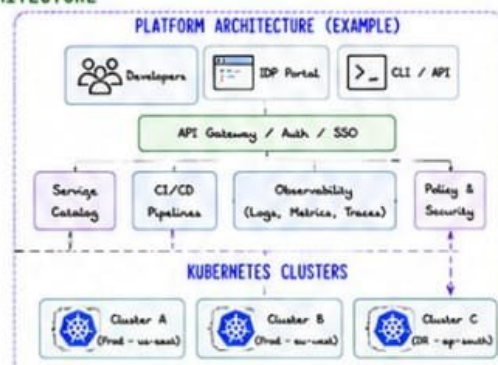
- Platform teams build and operate the IDP.
- They are product oriented.
- Focus on reliability, developer experience and continuous improvement.
- Enable, don't block.



Goal: Make the complex simple for developers.

6 PRODUCTION ARCHITECTURE

- Highly available, scalable and secure.
- Multi-cluster for isolation and resilience.
- Centralized services with decentralized execution.
- Observability and policies everywhere.



"Platform Engineering is building the runway so developers can fly."

19. ADVANCED KUBERNETES CHEAT SHEET

“ Know the commands. Master the cluster. ”

1 KUBECTL ADVANCED COMMANDS

\$ kubectl get all -A	# Get all resources in all namespaces
\$ kubectl get pods -A -o wide	# Get pods with node info
\$ kubectl describe pod <pod-name> -n <ns>	# Detailed pod info
\$ kubectl logs <pod-name> -n <ns> -c <container> --tail=100 -f	# Stream last 100 logs
\$ kubectl exec -it <pod-name> -n <ns> -- /bin/bash	# Execute shell in pod
\$ kubectl top nodes	# Show node metrics
\$ kubectl top pods -A	# Show pod metrics
\$ kubectl get events -A --sort-by=.metadata.creationTimestamp	# Get recent events
\$ kubectl drain <node> --ignore-daemonsets --delete-emptydir-data	# Drain a node
\$ kubectl uncordon <node>	# Uncordon a node
\$ kubectl port-forward svc/<svc-name> <local-port>:<port> -n <ns>	# Port forward service
\$ kubectl proxy	# Run kubectl proxy
\$ kubectl api-resources	# List all API resources
\$ kubectl explain <resource>	# Explain resource fields
\$ kubectl get <resource> -o yaml > output.yaml	# Export resource YAML
\$ kubectl diff -f <file-or-dir>	# Preview changes (server-side diff)
\$ kubectl apply --server-side -f <file>	# Server-side apply
\$ kubectl wait --for=condition=ready pod/<pod-name> --timeout=120s	# Wait for pod ready



2 HELM COMMANDS

\$ helm search repo <keyword>	# Search charts
\$ helm repo add <name> <url>	# Add chart repo
\$ helm repo update	# Update repo index
\$ helm install <release> <chart> -n <ns>	# Install chart
\$ helm install <release> <chart> -n <ns> -f values.yaml	# Install with values
\$ helm upgrade <release> <chart> -n <ns> -f values.yaml	# Upgrade release
\$ helm rollback <release> <revision> -n <ns>	# Rollback release
\$ helm list -A	# List all releases
\$ helm status <release> -n <ns>	# Release status
\$ helm history <release> -n <ns>	# Release history
\$ helm uninstall <release> -n <ns>	# Uninstall release
\$ helm template <release> <chart> -f values.yaml	# Render templates locally
\$ helm link <chart-path>	# Link chart
\$ helm dependency update <chart-path>	# Update chart dependencies



3 KUSTOMIZE COMMANDS

\$ kustomize build .	# Build kustomization
\$ kustomize build overlays/dev	# Build specific overlay
\$ kustomize edit set image <name>=<new-image>	# Set image
\$ kustomize edit add resource <file.yaml>	# Add resource
\$ kustomize edit add patch <patch.yaml>	# Add patch
\$ kustomize cfg tree	# Show kustomize tree
\$ kustomize cfg grep <keyword>	# Search in configs



 Kustomize = Template-free way to customize Kubernetes YAML. Perfect for overlays and environment management.

4 RBAC COMMANDS

\$ kubectl get roles -A	# List roles
\$ kubectl get rolebindings -A	# List rolebindings
\$ kubectl get clusterroles	# List clusterroles
\$ kubectl get clusterrolebindings	# List clusterrolebindings
\$ kubectl describe role <role-name> -n <ns>	# Describe role
\$ kubectl describe clusterrole <role-name>	# Describe clusterrole
\$ kubectl create role <role-name> --verb=get,list --resource=pods -n <ns>	# Create role
\$ kubectl create rolebinding <rb-name> --role=<role-name> \ --user=<user> -n <ns>	# Bind user to role
\$ kubectl create clusterrolebinding <rb-name> --user=<user>	# Bind user to clusterrole



 Use the principle of least privilege. Grant only what is necessary.

5 NETWORKING COMMANDS

\$ kubectl get svc -A	# List services
\$ kubectl get ep -A	# List endpoints
\$ kubectl get ingress -A	# List ingress
\$ kubectl describe svc <svc-name> -n <ns>	# Describe service
\$ kubectl describe ingress <ing-name> -n <ns>	# Describe ingress
\$ kubectl get networkpolicy -A	# List network policies
\$ kubectl describe networkpolicy <np-name> -n <ns>	# Describe network policy
\$ kubectl run netshoot --rm -it --image=calico/netshoot -- /bin/bash	# Debug networking
\$ kubectl exec -it <pod> -n <ns> -- nslookup <svc-name>	# DNS lookup
\$ kubectl exec -it <pod> -n <ns> -- curl <service-ip>:<port>	# Test connectivity



GENERAL TIPS

- ✓ Use -n <namespace> to specify namespace.
- ✓ Use -A for all namespaces.
- ✓ Use -o wide for more details.
- ✓ Use --dry-run/cient -o yaml to preview manifests.
- ✓ Use labels and selectors to filter resources.



6 QUICK REFERENCE TABLES

POD PHASES	
Pending	Pod accepted, waiting for scheduling
Running	Pod is running on a node
Succeeded	Pod completed successfully
Failed	Pod terminated with error
Unknown	Pod state unknown

RESTART POLICIES	
Always	Always restart the pod
OnFailure	Restart on failure (except 0 exit code)
Never	Never restart

SERVICE TYPES	
ClusterIP	Internal (default)
NodePort	Expose on each node IP
LoadBalancer	Expose via cloud LB
ExternalName	Map to external DNS name

RESOURCE SHORTCUTS		
RESOURCE	GET COMMAND	DESCRIPTION
Pods	kubectl get pods -A	List all pods
Deployments	kubectl get deploy -A	List all deployments
Services	kubectl get svc -A	List all services
ConfigMaps	kubectl get cm -A	List all configmaps
Secrets	kubectl get secrets -A	List all secrets
Nodes	kubectl get nodes	List all nodes
Namespaces	kubectl get ns	List all namespaces

“ Great engineers don't just use Kubernetes. They command it. ”

20. ADVANCED KUBERNETES MASTER CHECKLIST

“Check everything. Automate everything. Trust nothing.”

1 NETWORKING REVIEW

- ☐ CNI plugin is production ready and configured
- ☐ NetworkPolicies are defined and enforced
- ☐ Default deny policy is applied
- ☐ Ingress controller is highly available
- ☐ TLS is enabled for all ingress
- ☐ DNS is reliable and monitored
- ☐ Service mesh (if used) is healthy
- ☐ Egress policies and NAT rules reviewed
- ☐ Pod-to-pod connectivity validated
- ☐ Network observability enabled (Flows, latency)



★ Secure, reliable and observable networking is the foundation of Kubernetes.

2 SECURITY REVIEW

- ☐ RBAC is properly configured
- ☐ Least privilege access enforced
- ☐ ServiceAccounts are scoped
- ☐ Secrets are encrypted (etcd encryption)
- ☐ Image scanning enabled
- ☐ Pod Security Standards enforced
- ☐ Admission controllers active
- ☐ mTLS enabled (if using service mesh)
- ☐ Audit logging enabled
- ☐ Regular access review and cleanup



★ Security is not a feature. It's a continuous practice.

3 PLATFORM REVIEW

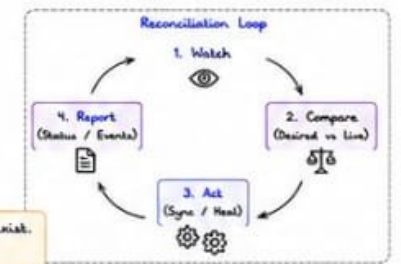
- ☐ Node groups are right-sized
- ☐ Resource requests/limits enforced
- ☐ Namespaces are organized
- ☐ Quotas and LimitRanges set
- ☐ Default resources defined
- ☐ Storage classes reviewed
- ☐ Backup and snapshots configured
- ☐ Monitoring and alerting in place
- ☐ Logging centralized
- ☐ Cost visibility enabled



★ A strong platform makes developers fast, safe and productive.

4 GITOPS REVIEW

- ☐ Git is the single source of truth
- ☐ Manifests are up-to-date
- ☐ Reconciliation is healthy
- ☐ Drift detection enabled
- ☐ Automated sync policies set
- ☐ RBAC for GitOps tools set
- ☐ Multi-cluster deployments working
- ☐ Secrets management integrated
- ☐ Rollback strategy tested
- ☐ Alerts configured for sync failures



★ If it's not in Git, it doesn't exist. Automate the entire lifecycle.

5 SERVICE MESH REVIEW

- ☐ Control plane is highly available
- ☐ mTLS enforced between services
- ☐ Traffic policies configured
- ☐ Retries, timeouts and circuit breaking set
- ☐ Observability (tracing/metrics) enabled
- ☐ Rate limiting configured
- ☐ Authorization policies enforced
- ☐ Gateway configuration reviewed
- ☐ Certificate rotation enabled
- ☐ Mesh performance monitored



★ Service mesh gives you visibility, security and control.

6 CLUSTER GOVERNANCE

- ☐ Cluster roles and bindings reviewed
- ☐ Namespaces have owners
- ☐ Policy as Code enforced
- ☐ Compliance controls in place
- ☐ Change management process defined
- ☐ Tagging and labeling standards
- ☐ Resource lifecycle policies set
- ☐ Access reviews are scheduled
- ☐ Documentation is up-to-date
- ☐ Governance dashboards in place



★ Governance aligns speed with responsibility. Make it visible and repeatable.

7 ENTERPRISE READINESS ASSESSMENT

- ☐ High availability validated
- ☐ Disaster recovery plan tested
- ☐ Backup and restore tested
- ☐ Multi-cluster strategy defined
- ☐ SLOs and SLIs defined
- ☐ Capacity planning in place
- ☐ Cost optimization reviewed
- ☐ Observability end-to-end
- ☐ Incident response playbooks ready
- ☐ Regular chaos testing performed



Reliability
Apps are resilient and self-healing



Security
Secure by default and continuously monitored



Performance
Scalable, efficient and well optimized



Operability
Observable, automated and easy to operate



Portability
Works across clouds and environments



Business Ready
Aligned with business goals and compliance



PRODUCTION READY!
✓ Tested
✓ Trusted
✓ Automated
✓ Repeatable



“Master the cluster. Automate the boring. Focus on building.”